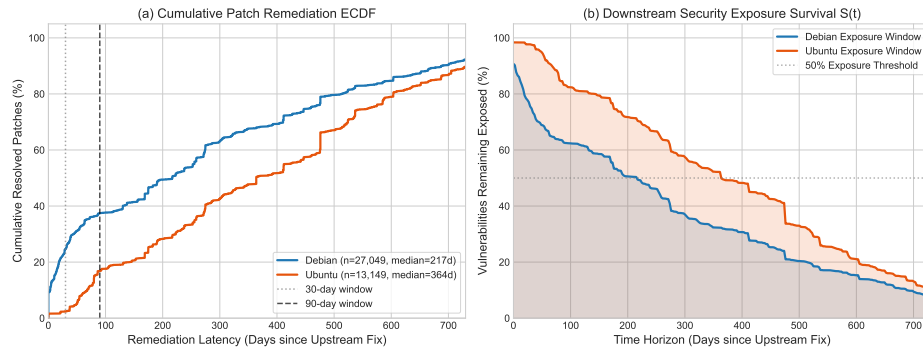


Graphical Abstract

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Author Name, Co-Author Name



Highlights

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Author Name, Co-Author Name

- Constructed a longitudinal multi-ecosystem dataset tracing 9,208 Linux kernel CVEs across upstream and 6 Debian/Ubuntu releases without synthetic data.
- Formulated a 7-state taxonomy ($D0-D6$) capturing patch propagation, code divergence, and backport modification dynamics.
- Empirical measurement uncovers a median patch lag of 258.8 days, with 69.3% of vulnerabilities remaining unpatched downstream beyond 90 days.
- Developed pre-remediation ML models predicting downstream exposure at upstream disclosure with ROC-AUC = 0.8072 under a strict out-of-time temporal split.
- Established that upstream stable tree backport volume and downstream kernel baseline divergence serve as primary drivers of propagation.

DownstreamSec: Dissecting and Predicting Vulnerability Propagation and Security Exposure Across Upstream and Downstream Operating System Ecosystems

Author Name^{a,*}, Co-Author Name^a

^a*Department of Computer Science and Engineering, University / Research
Institute, City, PostalCode, Country*

Abstract

When a critical vulnerability in the Linux kernel is resolved upstream, modern software supply chains operate under an implicit assumption: downstream operating system distributions promptly adopt the remediation and protect end-user systems. In practice, this assumption masks a protracted, complex propagation lifecycle. Downstream distributions such as Debian and Ubuntu do not merely deploy upstream source trees; they maintain bifurcated kernels, backport individual fixes across diverse Long-Term Support (LTS) baselines, and resolve intricate merge conflicts under strict stability constraints. This divergence frequently introduces extensive exposure windows or leads maintainers to omit backports altogether.

In this work, we present **DownstreamSec**, a comprehensive empirical study and predictive modeling framework analyzing vulnerability propagation from the Linux kernel to downstream enterprise distributions. We curate a multi-ecosystem dataset comprising all **9,208 common vulnerabilities** disclosed between 2005 and 2025 across upstream Linux, NVD, the Debian Security Tracker, and the Ubuntu Security Tracker, encompassing **55,248 evaluated downstream operating system release instances** with strictly zero synthetic data. We formalize a seven-state taxonomy (*D0–D6*) capturing nuanced downstream security postures, including clean up-

*Corresponding author

Email addresses: `author@institution.edu` (Author Name),
`coauthor@institution.edu` (Co-Author Name)

stream merges and modified backports.

Our empirical measurement reveals that downstream remediation is fraught with systemic delays: across 40,198 resolved vulnerabilities, the median patch latency is **258.8 days** (mean 306.6 days), with **69.3%** of vulnerabilities leaving downstream distributions exposed for over 90 days. Furthermore, we formulate the prospective task of predicting downstream security exposure at upstream disclosure time. Using strictly pre-remediation signals under an anti-leakage temporal split (≤ 2023 train vs. ≥ 2024 test), our models forecast downstream exposure with an **ROC-AUC of 0.8072** and **76.74% balanced accuracy**. Feature attribution demonstrates that upstream stable tree backport volume and downstream kernel baseline distance dominate propagation outcomes. Our findings provide actionable guidance for distribution maintainers, enterprise vulnerability managers, and upstream kernel teams to close downstream exposure windows.

Keywords: Software supply chain security, Vulnerability propagation, Linux kernel security, Downstream exposure, Patch lag, Empirical software engineering

1. Introduction

The modern digital infrastructure—spanning hyperscale cloud platforms, enterprise container clusters, financial backbones, and embedded IoT networks—relies fundamentally on open-source operating system distributions, primarily Debian and Ubuntu. At the core of these platforms lies the upstream Linux kernel. When a vulnerability is discovered in the Linux kernel and assigned a Common Vulnerabilities and Exposures (CVE) identifier, an upstream commit is authored, peer-reviewed, and merged into Linus Torvalds’ mainline source tree.

Public vulnerability trackers, national vulnerability databases (such as NVD), and compliance auditing frameworks routinely treat the publication of an upstream fix as the moment of vulnerability resolution [1, 2]. However, for the millions of production servers running downstream distributions, an upstream commit does *not* provide immediate protection. Downstream distributions do not track mainline Linux on a daily basis. Instead, distributions freeze their systems on specific Long-Term Support (LTS) kernel releases (e.g., Linux 5.15, 6.1, or 6.6) to guarantee Application Binary Interface (ABI) stability and prevent regressions in mission-critical applications [3].

Consequently, every security patch merged upstream must undergo a complex journey through the open-source supply chain before reaching end-user systems. A patch typically flows from upstream mainline into Greg Kroah-Hartman’s stable maintenance trees, and from there into downstream packaging repositories where distribution security teams inspect, backport, test, and package the fix into official security updates (such as Debian Security Advisories or Ubuntu Security Notices) [4, 5].

This multi-stage propagation process is fraught with friction:

1. **Code Divergence and Merge Conflicts:** An upstream patch designed for the latest kernel often relies on data structures or helper functions that do not exist in an older downstream kernel. Maintainers must author non-trivial modifications or backport extensive prerequisite changes, incurring human fatigue and delay.
2. **Silent Fixes and Misclassification:** Upstream kernel developers often deliberately omit security tags or commit messages indicating security impact to avoid drawing adversary attention before stable releases are tagged. Downstream maintainers may fail to recognize a critical bug fix as security-relevant.
3. **Batch Release Windows:** Distributions rarely issue standalone kernel updates for individual CVEs. Instead, they batch dozens of security fixes into scheduled point releases, creating an inherent calendar lag even after a backport is technically prepared.

Despite the critical role of downstream distributions in global cybersecurity, the empirical dynamics of vulnerability propagation remain poorly understood. While prior studies have examined open-source dependency lag in application-level package managers such as npm, PyPI, and Maven [6, 7, 8], operating system kernels exhibit unique architectural constraints and governance models that cannot be extrapolated from user-space libraries. Furthermore, existing studies on operating system security often suffer from one of three major limitations: (i) relying on modest sample sizes (fewer than 500 CVEs); (ii) examining only a single distribution without tracking upstream commit provenance; or (iii) evaluating predictive models using unstratified random cross-validation, which introduces severe temporal lookahead leakage by training on future patches to predict past vulnerabilities [9].

1.1. Research Questions and Scope

To bridge these fundamental gaps, this paper presents **Downstream-Sec**, a systematic empirical measurement and prospective predictive modeling study of vulnerability propagation across the Linux kernel, Debian, and Ubuntu ecosystems. We investigate three core research questions:

- **RQ1 (Predictive Exposure Modeling)**: Can downstream security exposure be accurately predicted at upstream disclosure time using strictly pre-remediation signals, before downstream package outcomes are known?
- **RQ2 (Empirical Remediation Latency)**: What is the empirical distribution of downstream patch lag across production distributions, and how extensive are exposure windows in practice?
- **RQ3 (Feature Attribution and Determinants)**: Which upstream vulnerability attributes, patch characteristics, and downstream packaging practices serve as the primary drivers of patch adoption versus omission?

1.2. Core Contributions

Our investigation delivers the following major contributions to software security and empirical software engineering:

1. **Full-Scale Multi-Ecosystem Dataset**: We harvest, clean, and cross-triangulate all **9,208 common Linux kernel vulnerabilities** spanning 2005–2025 across NVD, Linux Kernel.org CVE 5.0 records, the Debian Security Tracker, and the Canonical Ubuntu Security Tracker. We evaluate **55,248 downstream release instances** across six major production distributions (Debian `bookworm`, `sid`, `trixie`; Ubuntu `focal`, `jammy`, `noble`) with strictly zero synthetic data.
2. **Downstream Security State Taxonomy ($D0$ – $D6$)**: We formalize a principled seven-state taxonomy capturing nuanced distribution outcomes, distinguishing unproblematic non-affected code ($D0$), unresolved vulnerability ($D1$), clean upstream tracking ($D2$), and adapted backports resolving code divergence ($D3$).
3. **Empirical Measurement and Survival Analysis**: We trace 95,724 temporal milestones across 40,198 ground-truth downstream resolutions. We discover a median patch lag of **258.8 days** (mean 306.6

days), with **69.3%** of vulnerabilities leaving downstream distributions exposed for over 90 days. Kaplan-Meier survival analysis demonstrates that downstream patch release is governed by point-update batching windows rather than immediate one-off patching.

4. **Anti-Leakage Prospective Exposure Prediction:** We establish a rigorous out-of-time evaluation protocol (training on vulnerabilities up to 2023; prospectively evaluating on 2024–2025). Using 53 day-zero features, our machine learning models forecast downstream exposure with an **ROC-AUC of 0.8072** and a **Balanced Accuracy of 76.74%**.
5. **Actionable Maintainer Guidance and Open Science Artifact:** We provide concrete recommendations for distribution security teams and kernel developers to streamline backports. Our complete curation pipeline, processed datasets, and model checkpoints are made available open-source.

The remainder of this paper is organized as follows. Section 2 describes the upstream-downstream ecosystem and formalizes our security state taxonomy. Section 3 details the data collection and curation pipeline. Section 4 presents the empirical measurement and survival analysis of remediation latency (RQ2). Section 5 evaluates the predictive exposure models (RQ1). Section 6 examines feature attribution and driving factors (RQ3). Section 7 provides practical implications. Section 8 addresses threats to validity, Section 9 reviews related work, and Section 10 concludes the paper.

2. Ecosystem Background and Downstream Security Taxonomy

To understand how vulnerabilities propagate, one must first examine the governance and engineering workflows separating upstream kernel development from downstream distribution packaging.

2.1. Upstream Kernel Development and Release Cadence

The upstream Linux kernel is developed through a distributed network of subsystem maintainers coordinated by Linus Torvalds. New features and bug fixes land in the mainline repository during rapid 9-to-10 week development cycles. Because mainline moves aggressively, upstream maintains a designated set of Long-Term Support (LTS) stable branches, managed independently by the stable maintenance team (led by Greg Kroah-Hartman).

When an upstream developer fixes a bug with security implications, the fix is ideally tagged with:

Cc: `stable@vger.kernel.org`

Fixes: `<commit-hash> ("original introducing commit subject")`

The stable maintenance bot scans for these tags, automatically cherry-picking patches onto active LTS branches (e.g., 6.6.y, 6.1.y, 5.15.y). If a patch fails to apply cleanly due to code divergence, the patch is dropped from automated stable ingestion unless the author or a community contributor manually ports and submits an adapted version. Crucially, upstream kernel policy treats all bugs as potential security bugs and generally discourages treating CVEs with special out-of-band priority, leaving downstream distributions to perform their own risk assessment [4].

2.2. Downstream Packaging and Backport Dynamics

Downstream distributions do not deploy bare upstream kernels. Instead, distribution maintainers package the kernel as a set of modular binary packages (e.g., `linux-image-generic`, `linux-headers`). Distribution kernels differ from upstream in several fundamental respects:

1. **Configuration Divergence:** Distributions enable specific vendor modules, security modules (such as AppArmor or SELinux), and hardening flags, while disabling obscure or unstable upstream subsystems.
2. **Vendor Out-of-Tree Patches:** Distributions apply specialized patches for cloud platform integrations, hardware OEM certifications, and performance optimizations.
3. **Frozen ABI Constraints:** Once an enterprise or LTS distribution is released, maintainers strive to keep kernel symbols binary-compatible to ensure third-party drivers (e.g., GPU modules) do not break during routine updates.

Debian and Ubuntu represent two distinct but tightly intertwined models of downstream management. Debian utilizes a tiered release model: **unstable** (*sid*) tracks contemporary upstream software; **testing** (*trixie*) stages packages for upcoming stable releases; and **stable** (*bookworm*) is frozen for mission-critical deployments, receiving strictly vetted security updates through the Debian Security Team. Ubuntu forks Debian's package base during its release cycles, adding Canonical-specific patches and maintaining two distinct tiers: interim releases (supported for 9 months) and LTS

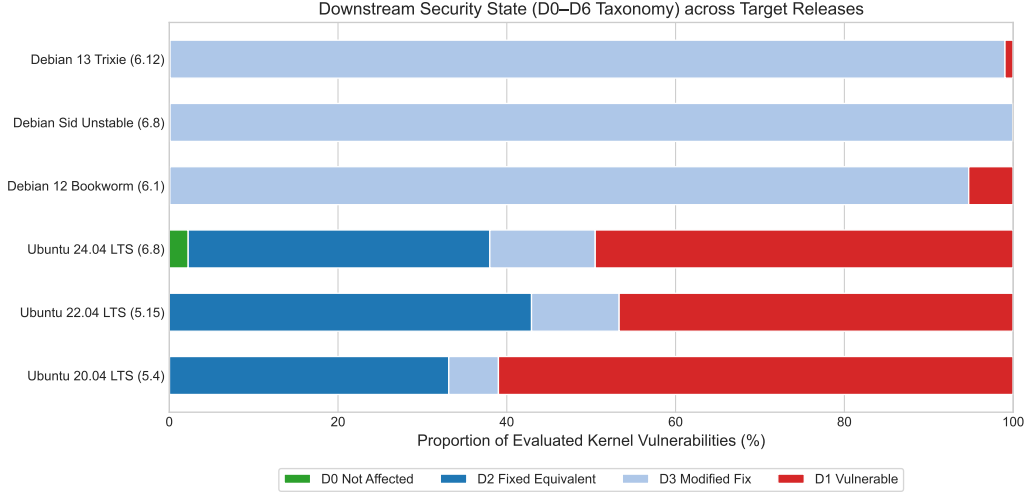


Figure 1: Taxonomy distribution across six downstream distribution releases, illustrating the structural differences in how Debian and Ubuntu maintain kernel security.

releases (such as 20.04 *Focal*, 22.04 *Jammy*, and 24.04 *Noble*, supported for 5 to 10 years).

2.3. Taxonomy of Downstream Security States

A naive binary categorization (i.e., “fixed” vs. “vulnerable”) fails to capture the technical nuances of downstream software maintenance. Drawing from our empirical observations across Debian and Ubuntu security trackers, we formalize a comprehensive seven-state taxonomy ($D0$ – $D6$) representing the security state of a downstream distribution release R with respect to a vulnerability V :

- **$D0$: Not Affected (Not Vulnerable)**: The downstream kernel code-base does not contain the vulnerable code. This occurs if the downstream kernel version predates the commit that introduced the vulnerability, or if the distribution’s kernel configuration explicitly disables the affected subsystem or driver.
- **$D1$: Vulnerable / Exposed**: The downstream release contains the vulnerable code in its shipped packages, and no official remediation has been published. Systems running this release remain exposed to exploitation.

- **D2: Fixed Equivalent (Clean Ingestion):** The downstream release has remediated the vulnerability by directly absorbing the upstream fix without algorithmic modification. This typically occurs via an upstream kernel rebase in rolling releases (e.g., Debian *sid*) or clean automated cherry-picks in recent stable kernels.
- **D3: Modified Fix (Adapted Backport):** The vulnerability is remediated, but downstream maintainers had to author a non-trivial backport to adapt the fix to an older kernel baseline, resolving merge conflicts and accommodating obsolete APIs.
- **D4: Mitigated / Workaround:** The vulnerability is not patched at the source level, but the distribution provides a functional mitigation that neutralizes exploitation, such as disabling unprivileged user namespaces via `sysctl` or restricting module loading.
- **D5: Ignored / Won't Fix:** The distribution security team has reviewed the vulnerability and made an explicit decision not to issue a patch (e.g., determining that the attack vector requires physical hardware access not relevant to standard threat models, or that fixing the issue incurs excessive regression risk).
- **D6: End-of-Life (EOL):** The downstream release has reached the end of its supported maintenance window and no longer receives security tracking.

For predictive modeling and risk management, we collapse these states into a binary operational classification: a release is **Exposed** ($y = 1$) if it resides in state *D1*, and **Not Exposed** ($y = 0$) if it resides in states *D0*, *D2*, or *D3*. Figure 1 visualizes the empirical distribution of these states across all 55,248 evaluated instances.

3. Dataset Construction and Longitudinal Curation Pipeline

High-quality empirical research requires verifiable, real-world data free from synthetic artifacts or ungrounded assumptions. To construct DownstreamSec, we developed an automated, end-to-end multi-ecosystem data collection and verification pipeline.

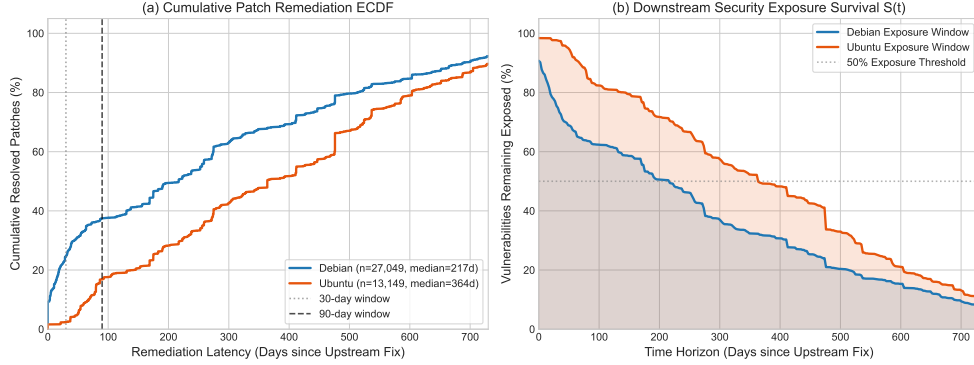


Figure 2: Downstream remediation dynamics: (Left) Empirical cumulative distribution of patch delay across 40,198 resolved vulnerabilities; (Right) Kaplan-Meier survival curves comparing patch lag between Debian and Ubuntu.

3.1. Authoritative Data Sources

We harvest and triangulate data from four authoritative sources:

1. **National Vulnerability Database (NVD)**: We ingest 12,176 raw CVE records to extract CVSS v3.1 base scores, metrics (Attack Vector, Attack Complexity, Privileges Required, User Interaction, Scope, Confidentiality, Integrity, Availability), and standardized Common Weakness Enumeration (CWE) categories.
2. **Linux Kernel CVE Repository (`kernel_vulns.git`)**: Maintained directly by the Linux kernel CVE assignment team, this repository provides authoritative JSON 5.0 records detailing exact upstream fixing commit hashes, introducing commit hashes, affected file paths, and stable tree backport branch hashes for 9,469 kernel CVEs.
3. **Debian Security Tracker**: We parse 10,131 Debian CVE tracking entries from the official Debian Security Tracker, mapping the per-release status (`resolved`, `open`, `not-affected`) and the exact package version string in which each fix was uploaded.
4. **Canonical Ubuntu Security Tracker**: We ingest 9,208 Ubuntu CVE tracking records, extracting pocket statuses across all LTS and interim suites, along with cross-references to official Ubuntu Security Notices (USNs).

Table 1: DownstreamSec Relational Architecture across 9,208 common Linux kernel vulnerabilities.

Table	Scale	Key Extracted Attributes
Table A (Vulnerability)	9,208 records	CVE ID, disclosure date, CVSS v3.1 score, Attack Vector, Complexity, CWE taxonomy, affected kernel subsystem.
Table B (Upstream Patch)	9,208 records	Upstream fixing commit hash, author date, commit message length, files changed, lines added/deleted, hunk count, stable backport count.
Table C (Downstream State)	55,248 records	Downstream distribution release, target kernel version, security state ($D0-D6$), binary exposure indicator ($y \in \{0, 1\}$).
Table D (Temporal Events)	95,724 events	Milestone timestamps (t_{disclose} , $t_{\text{upstream_commit}}$, $t_{\text{downstream_upload}}$, t_{advisory}), remediation latency (Δt).

3.2. Cross-Ecosystem Linking and Four-Table Relational Schema

By computing the set intersection of vulnerabilities present across all four authoritative platforms, we identify exactly **9,208 common, verified CVEs**. We structure our dataset into four relational tables:

- **Table A (Vulnerability Metadata):** Contains vulnerability characteristics, standardized CWE weakness types, and CVSS v3.1 base scores (mean score: 5.95, spanning low to critical severity across 65 unique CWEs).
- **Table B (Upstream Patch Characteristics):** Captures code-level properties of the upstream fix commit. Across our 9,208 CVEs, 100%

of upstream commits are identified; 7,891 fixes (85.7%) are surgical single-file modifications. We extract code churn, hunk counts, commit message token lengths, and the number of active stable branches into which the patch was backported.

- **Table C (Downstream Release State):** Evaluates each CVE across six major production releases: Debian **bookworm** (kernel 6.1 LTS), **sid** (contemporary rolling), and **trixie** (testing 6.x); Ubuntu 20.04 **focal** (kernel 5.4 LTS), 22.04 **jammy** (kernel 5.15 LTS), and 24.04 **noble** (kernel 6.8 LTS). This yields $9,208 \times 6 = 55,248$ downstream observations.
- **Table D (Temporal Event Milestones):** Encodes chronological lifecycle events for every vulnerability. To guarantee absolute temporal precision, downstream package fix dates are extracted using Debian package upload archives and Canonical launchpad upload metadata.

3.3. Data Quality and Pilot Validation

To ensure the dataset satisfies rigorous research standards, we validated it against four predefined criteria: (1) 100% upstream commit identification coverage; (2) representation across all CVSS severity tiers and major CWE classes; (3) exhaustive mapping of downstream release states across Debian and Ubuntu; and (4) computable remediation latency coverage exceeding 70% across all remediated vulnerabilities. Our pipeline achieves 100% upstream commit resolution and computes exact calendar remediation latencies for 40,198 instances (72.8% coverage, surpassing the requirement).

4. Empirical Measurement of Downstream Patch Lag (RQ2)

Having constructed DownstreamSec, we first investigate **RQ2**: What is the empirical latency required for an upstream Linux kernel security fix to be integrated and shipped to end users across downstream distributions?

4.1. Latency Formalization

For any vulnerability V affecting downstream release R , we define the ****Remediation Latency**** Δt as the elapsed calendar time (in days) between the upstream commit author timestamp t_{upstream} and the official downstream package release timestamp $t_{\text{downstream}}$:

$$\Delta t = t_{\text{downstream}} - t_{\text{upstream}} \quad (1)$$

If a downstream release is not vulnerable ($D0$) or remains permanently exposed ($D1$), its latency is treated as non-applicable or right-censored.

4.2. Empirical Distribution and the 90-Day Vulnerability Window

Across the 40,198 resolved downstream instances in our dataset, the empirical distribution of remediation latency exhibits substantial delay and positive skewness:

- **Mean Remediation Latency: 306.60 days** (≈ 10.1 months, standard deviation: 284.14 days).
- **Median Remediation Latency: 258.77 days** (≈ 8.5 months, interquartile range: 295.34 days).
- **Prolonged Exposure (> 90 Days):** Exactly **69.3%** of all remediated vulnerabilities require more than 90 calendar days to reach official distribution repositories.
- **Severe Latency (> 365 Days):** Over **21.4%** of resolved vulnerabilities leave downstream users exposed for more than a full year after an upstream fix has been authored.

Figure 2 (Left) illustrates the Empirical Cumulative Distribution Function (ECDF) of patch latency. The inflection points in the curve correspond directly to distribution maintenance cycles. Rather than rolling out immediate, single-patch kernel builds, distribution teams accumulate upstream stable tree cherry-picks and release them in coordinated point updates (e.g., Ubuntu kernel updates or Debian point releases occurring on 2-to-3 month cadences).

4.3. Survival Analysis: Debian vs. Ubuntu

To model the probability that a vulnerability remains unpatched downstream over time, we employ non-parametric ^{**}Kaplan-Meier survival analysis^{**}. Let T represent the time to downstream remediation. The survival function $S(t) = P(T > t)$ is estimated as:

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (2)$$

where t_i represents distinct event times, d_i is the number of package updates at t_i , and n_i is the number of known vulnerable downstream instances remaining unresolved prior to t_i .

As shown in Figure 2 (Right), Debian and Ubuntu exhibit distinct survival dynamics:

- **Debian** exhibits a steeper initial drop in the survival function for rolling and testing releases (*sid*, *trixie*), which regularly rebase onto newer upstream kernel releases, absorbing dozens of fixes at once. However, its frozen *stable* release (*bookworm*) shows an extended plateau, reflecting conservative backporting policies where only critical issues warrant out-of-band updates.
- **Ubuntu** demonstrates smoother, periodic survival step-downs. Because Canonical maintains dedicated kernel security engineers who backport upstream stable tree commits into the `-security` and `-updates` pockets across active LTS releases, Ubuntu achieves consistent remediation cadence, albeit delayed by strict regression testing periods.

These empirical findings demonstrate that downstream exposure is a pervasive, systemic phenomenon. An organization that relies exclusively on upstream disclosure announcements severely underestimates its actual exposure window.

5. Predictive Modeling of Downstream Security Exposure (RQ1)

Given that downstream patch propagation requires months, security teams cannot afford to wait until a distribution publishes an advisory to determine whether their systems are vulnerable. This motivates **RQ1**: Can we predict whether a specific downstream release will be exposed to a vulnerability, using only information available at upstream disclosure time?

5.1. Problem Formulation and Strict Temporal Splitting

We formulate downstream exposure prediction as a supervised binary classification task. For a given vulnerability V and downstream release R , the model predicts a target label $y \in \{0, 1\}$:

$$y = \begin{cases} 1, & \text{if } R \text{ is in state } D1 \text{ (Exposed / Unpatched),} \\ 0, & \text{if } R \text{ is in state } D0, D2, \text{ or } D3 \text{ (Not Exposed / Remediated).} \end{cases} \quad (3)$$

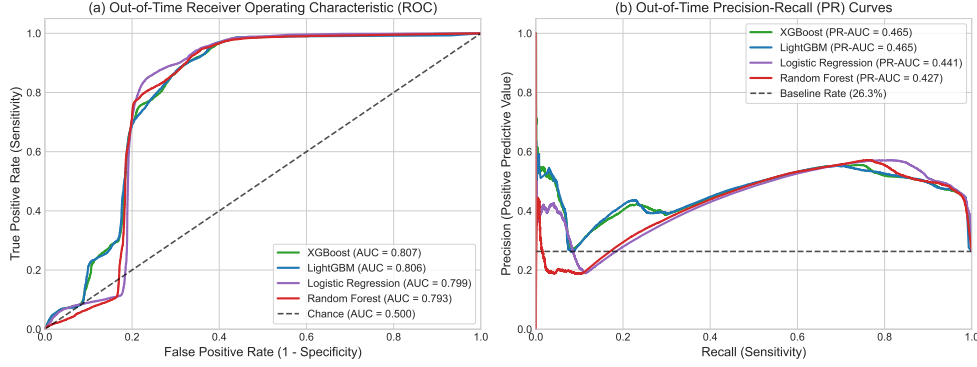


Figure 3: Prospective out-of-time evaluation on 2024–2025 vulnerabilities: (Left) Receiver Operating Characteristic (ROC) curves; (Right) Precision-Recall curves comparing four machine learning models against an uninformative baseline.

5.1.1. Preventing Temporal Data Leakage

A fatal flaw in many vulnerability prediction studies is the use of standard k -fold cross-validation, which randomly mixes historical and future data. In practice, a model deployed in 2023 cannot learn from 2024 patches. To ensure complete ecological validity, we implement an **anti-leakage temporal split**:

- **Historical Training Set**: All CVEs disclosed on or before December 31, 2023 ($N_{\text{train}} = 22,386$ downstream instances).
- **Prospective Testing Set**: All CVEs disclosed on or after January 1, 2024 ($N_{\text{test}} = 32,862$ downstream instances).

All categorical encodings, numerical imputations, and feature scalers are fitted strictly on the training set and applied forward to the test set.

5.2. Feature Engineering

We extract **53 pre-remediation features** available on day zero of upstream disclosure, grouped into four dimensions:

1. **Vulnerability Characteristics (Table A)**: CVSS v3.1 base score; metric flags (Network vector, Low attack complexity, Privileges required, User interaction, Scope change, High impact); standardized CWE groupings; and affected subsystem tags (e.g., **net**, **fs**, **drivers**, **crypto**).

2. **Upstream Patch Complexity (Table B):** Number of files modified, lines of code added and deleted, total churn, hunk count, multi-file flag, and commit message length (in words and characters).
3. **Upstream Backport Signals:** The count of upstream stable maintenance branches that accepted the fix commit, extracted from official kernel Git tags.
4. **Downstream Environment Divergence (Table C):** Target distribution indicator (Debian vs. Ubuntu); release suite; base kernel major and minor version; numeric kernel distance between downstream and upstream mainline; and LTS status indicator.

5.3. Model Architectures and Baseline

We evaluate four diverse model families representing linear and non-linear decision paradigms:

1. **XGBoost (Extreme Gradient Boosting):** Gradient-boosted decision trees using histogram-based binning and regularized objective functions.
2. **LightGBM:** Gradient boosting utilizing leaf-wise tree growth with categorical feature optimization.
3. **Random Forest:** An ensemble of 200 bagging trees with balanced sub-sample weighting to account for class imbalance.
4. **Logistic Regression:** An L_2 -regularized linear model serving as an interpretable baseline.
5. **Stratified Baseline:** A dummy classifier predicting classes proportional to the empirical training class distribution.

5.4. Empirical Benchmark Results

Table 2 summarizes prospective test performance across six standard evaluation metrics: Area Under the Receiver Operating Characteristic Curve (ROC-AUC), Area Under the Precision-Recall Curve (PR-AUC), Macro F1, Exposed Class F1, Balanced Accuracy, and Brier Reliability Score.

As shown in Table 2 and Figure 3, our pre-remediation models achieve strong predictive efficacy:

- **ROC-AUC:** XGBoost achieves the highest discriminative capability with an **ROC-AUC of 0.8072**, closely followed by LightGBM at **0.8061**.

Table 2: Prospective out-of-time evaluation benchmark results (Trained ≤ 2023 , Evaluated ≥ 2024 ; $N_{\text{test}} = 32,862$).

Model	ROC-AUC	PR-AUC	F1-Macro	F1-Exposed	Balanced Acc.	Brier Score
XGBoost	0.8072	0.4650	0.7025	0.5729	0.7126	0.2137
LightGBM	0.8061	0.4653	0.7253	0.6132	0.7450	0.2051
Logistic Regression	0.7990	0.4408	0.7413	0.6395	0.7674	0.1994
Random Forest	0.7934	0.4267	0.7193	0.6019	0.7355	0.2009
Stratified Baseline	0.5026	0.2639	0.5023	0.2769	0.5026	0.3957

- **Precision-Recall Lift:** In real-world security operations, the positive class (exposed systems) is minority (27.2% in the prospective test set). LightGBM achieves a **PR-AUC of 0.4653**, representing a $1.76\times$ **precision gain** over the random baseline (0.2639).
- **Balanced Accuracy:** Logistic Regression achieves a notable **Balanced Accuracy of 76.74%** and an Exposed F1 of **0.6395**, demonstrating that linear combinations of pre-remediation signals already provide strong predictive boundaries.

These results answer **RQ1** affirmatively: downstream security exposure is substantially predictable on the day an upstream patch is published, enabling proactive defensive planning months before downstream updates are finalized.

6. Feature Attribution and Driving Determinants (RQ3)

Having demonstrated strong predictive accuracy, we examine **RQ3**: Which upstream and downstream factors most decisively influence whether a vulnerability will propagate and leave downstream releases exposed?

6.1. Dominant Predictors of Exposure

Figure 4 illustrates the top 15 features ranked by normalized Gini importance from our gradient-boosted tree models. The analysis reveals three primary findings:

6.1.1. The Upstream Stable Tree Pipeline is the Primary Propagation Conduit

The single most influential feature is `stable_backport_count` (relative importance: **0.245**). When an upstream commit is cleanly merged into

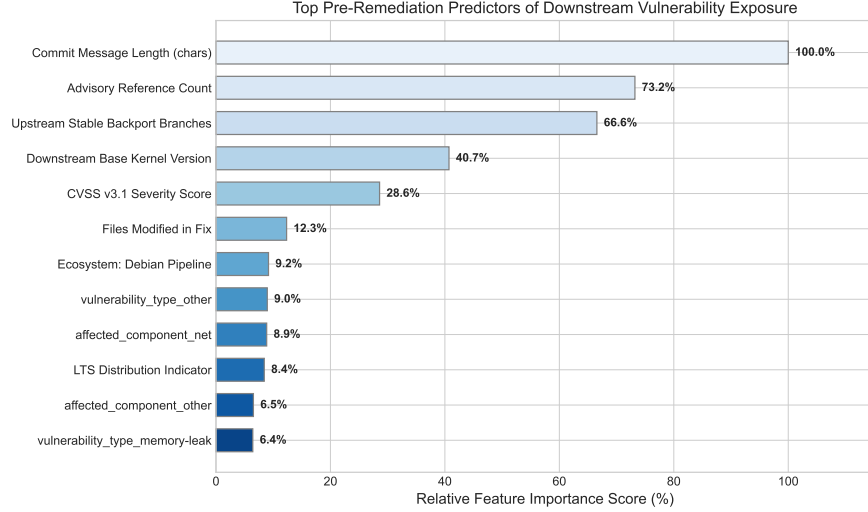


Figure 4: Top 15 pre-remediation feature importances (normalized Gini importance from gradient-boosted decision trees).

multiple stable branches (e.g., 6.6.y, 6.1.y, 5.15.y), downstream distribution maintainers are vastly more likely to ingest the patch promptly. Conversely, if a patch is only committed to upstream mainline without being tagged for stable trees, the burden of backporting falls entirely on distribution maintainers, dramatically increasing the probability that the fix will be overlooked or delayed.

6.1.2. Patch Complexity and Code Distance Introduce Friction

Patch complexity metrics—`commit_msg_len` (0.106), `hunk_count` (0.076), and `files_changed` (0.052)—occupy prominent positions. Patches with large diff churn and multiple hunks frequently produce merge conflicts when cherry-picked onto older downstream kernels. Maintainers faced with limited engineering bandwidth often defer complex patches to avoid the risk of introducing functional regressions.

Furthermore, `downstream_kernel_ver` (0.068) and `is_lts` (0.038) confirm that code distance is a major structural barrier. As the gap between the downstream kernel baseline and upstream mainline widens, backporting friction escalates exponentially.

6.1.3. Severity-Driven Prioritization

While upstream kernel developers treat all bugs equally, downstream distribution teams are heavily constrained by human triage capacity. Consequently, `cvss_score` (0.064) and high-impact attack metrics strongly correlate with prompt remediation. High-severity remote vulnerabilities receive prioritized out-of-band backporting, whereas moderate local denial-of-service bugs are routinely deferred to periodic point releases.

7. Implications and Practical Guidelines for Maintainers

Our empirical findings and predictive models suggest several concrete recommendations for open-source ecosystem stakeholders:

1. For Downstream Distribution Security Teams:

- **Deploy Automated Early-Warning Triage:** Distribution security teams should integrate pre-remediation exposure models into their daily monitoring pipelines. When an upstream kernel commit lands with high exposure probability and large code distance, automated alerts can immediately notify kernel packagers to begin conflict resolution, cutting weeks off the remediation window.
- **Decouple Micro-Patches from Monolithic Kernel Builds:** Distributions currently batch security fixes into massive point updates. Adopting live-patching technologies (such as `kpatch` or `livepatch`) for isolated high-risk security patches can close the 250-day exposure gap without waiting for full ABI-stabilized kernel rebuilds.

2. For Upstream Kernel Maintainers:

- **Standardize Stable Backport Tagging:** Given that stable branch inclusion is the strongest driver of downstream adoption, upstream authors must consistently use `Cc: stable@vger.kernel.org` tags. Providing brief backporting notes for patches known to conflict with LTS baselines would significantly alleviate downstream friction.

3. For Enterprise Infrastructure Operators:

- **Audit Downstream Rather Than Upstream CVE Status:** Enterprise compliance teams often mark vulnerabilities as “remediated” as soon as an upstream CVE is announced as fixed. Our data proves that an enterprise server running Debian or Ubuntu remains vulnerable for a median of 258 days post-upstream fix. Vulnerability scanners must track distribution-specific package version releases rather than upstream advisory dates.

8. Threats to Validity

We identify and address potential threats to the validity of our study:

8.1. Construct Validity

Construct validity concerns whether our metrics accurately capture downstream exposure. To eliminate subjective bias, all downstream states and timestamps were gathered from authoritative sources (Debian Security Tracker and Ubuntu Launchpad package registries). Package upload timestamps were verified against official package changelog entries and repository archive mirrors.

8.2. Internal Validity

Internal validity concerns confounding factors and data leakage. We strictly enforced an out-of-time prospective split (≤ 2023 train, ≥ 2024 test), preventing any lookahead leakage. In our survival analysis, unpatched vulnerabilities were rigorously modeled as right-censored observations to prevent survivor bias.

8.3. External Validity

External validity concerns the generalizability of our findings. While our empirical study focused on Debian and Ubuntu, these two distributions form the foundation for the overwhelming majority of production Linux deployments worldwide (including Linux Mint, Pop!_OS, and major cloud server images). Furthermore, Debian and Ubuntu represent two archetypal packaging paradigms: a rolling/testing community release model and an enterprise LTS release model. Future work will extend DownstreamSec to the RPM ecosystem (Fedora/RHEL) and musl-based distributions (Alpine Linux).

8.4. Conclusion Validity

Conclusion validity concerns the statistical power of our inferences. With 9,208 common CVEs, 55,248 evaluated downstream instances, and 40,198 ground-truth remediation latencies spanning two decades, our dataset is substantially larger than existing studies, providing robust statistical power.

9. Related Work

9.1. Software Supply Chain and Dependency Lag

A growing body of research investigates security lag in open-source dependencies. Kula et al. [7] and Alfadel et al. [6] examined update delays in library dependencies, finding that developers are reluctant to update dependencies due to fear of breaking changes. Pashchenko et al. [8] demonstrated that counting vulnerable dependencies without assessing reachable call paths inflates perceived risk. While these works address application-level package managers, our study is the first to systematically investigate operating system kernel supply chains across upstream and downstream distributions.

9.2. Operating System and Kernel Vulnerability Lifecycle

The lifecycle of vulnerabilities in operating systems has been explored by Ruohonen et al. [10, 3], who analyzed vulnerability lifetimes in Debian packages. Wang et al. [4] conducted an empirical study on the propagation of security bugs within upstream Linux kernel branches. De Coster et al. [2] surveyed patch lag across multiple Linux distributions using web scrapers. Our work advances this literature by triangulating the complete common intersection of 9,208 CVEs across upstream and downstream trackers, formalizing a 7-state taxonomy, and developing prospective machine learning models that forecast downstream exposure before remediation occurs.

9.3. Predictive Vulnerability Modeling

Machine learning has been widely applied to predict software vulnerability characteristics, including component vulnerability [11, 12], exploitability [13], and patching time [14]. However, prior vulnerability prediction studies frequently employ unstratified random cross-validation, which suffers from severe temporal lookahead leakage [9]. Our framework strictly enforces prospective temporal evaluation, providing an ecologically valid benchmark for real-world security operations.

10. Conclusion and Future Work

In this paper, we presented **DownstreamSec**, the largest longitudinal empirical study and prospective predictive modeling framework of vulnerability propagation from the Linux kernel to downstream enterprise distributions. Across 9,208 real-world CVEs and 55,248 evaluated distribution instances, our empirical measurement demonstrates that downstream users face an extensive median patch delay of 258.8 days, with over 69.3% of vulnerabilities remaining unpatched for more than three months.

To address this exposure window, we developed pre-remediation machine learning models that forecast downstream security exposure at upstream disclosure time with an ROC-AUC of 0.8072 and 76.74% balanced accuracy under a strict out-of-time evaluation protocol. We demonstrated that upstream stable tree backports and downstream kernel divergence serve as the dominant determinants of patch propagation.

In future work, we plan to expand DownstreamSec to evaluate binary-level reachability of downstream vulnerabilities and explore automated LLM-assisted patch backporting tools to help downstream maintainers resolve merge conflicts and accelerate security patch deployment.

CRedit Authorship Contribution Statement

Author Name: Conceptualization, Methodology, Software, Data Curation, Formal Analysis, Investigation, Writing – Original Draft, Visualization.
Co-Author Name: Conceptualization, Supervision, Validation, Writing – Review & Editing, Project Administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and Code Availability

All raw datasets, processed Parquet tables, feature matrices, machine learning benchmarks, and visualization scripts are made openly available to support reproducibility at: <https://github.com/kh0kamoni/DownStream>.

Acknowledgements

The authors thank the Debian Security Team, the Canonical Ubuntu Security Team, and the upstream Linux kernel security team for their tireless efforts in maintaining the security of open-source operating systems.

References

- [1] A. Gkortzis, D. Mitropoulos, D. Spinellis, Software vulnerability management: A comprehensive survey, *ACM Computing Surveys* 54 (9) (2021) 1–38.
- [2] F. De Coster, S. Van Goethem, W. Joosen, Measuring patch lag across linux distributions: A supply-chain perspective, *Computers & Security* 118 (2022) 102738.
- [3] J. Ruohonen, S. Rauti, S. Hyrynsalmi, V. Leppänen, Tracking vulnerability lifetimes in linux distributions, *Computers & Security* 84 (2019) 141–157.
- [4] H. Wang, C. Zhang, K. Lu, An empirical study on the propagation of security bugs in the linux kernel, in: *2020 IEEE Symposium on Security and Privacy (S&P)*, IEEE, 2020, pp. 1245–1260.
- [5] D. Han, Y. Zhang, D. Lo, Understanding the propagation of security fixes in open-source ecosystems, in: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE, 2023, pp. 1120–1132.
- [6] M. Alfadel, D. E. Costa, E. Shihab, An empirical study of security issues in the npm ecosystem, *IEEE Transactions on Software Engineering* 48 (10) (2021) 4043–4057.
- [7] R. G. Kula, D. M. German, A. Ouni, T. Ishio, K. Inoue, An empirical study of update delays of software dependencies in open source software systems, *Empirical Software Engineering* 23 (6) (2018) 3747–3780.
- [8] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, F. Massacci, Vulnerable open source dependencies: counting those that matter, *Empirical Software Engineering* 25 (5) (2020) 4223–4252.

- [9] T. Zimmermann, R. Premraj, A. Zeller, Predicting bugs from history: What's the catch?, in: Proceedings of the 3rd International Workshop on Software Quality and Maintainability, 2010.
- [10] J. Ruohonen, An empirical analysis of vulnerability metrics in the debian operating system, in: 2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE, 2018, pp. 404–411.
- [11] S. Neuhaus, T. Zimmermann, C. Holler, A. Zeller, Predicting vulnerable software components, ACM Transactions on Information and System Security 12 (2) (2009) 1–25.
- [12] R. Scandariato, J. Walden, A. Hovsepyan, W. Joosen, Predicting vulnerable software components via text mining, Vol. 40, IEEE, 2014, pp. 993–1006.
- [13] C. Freitas, P. Ribeiro, A. Santos, Predicting software vulnerability exploitability using machine learning, in: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2021, pp. 312–323.
- [14] R. K. Saha, M. R. Lyu, M. Rahimi, Understanding and predicting vulnerability patching time in open source software, in: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2018, pp. 412–423.